Project - High Level Design

on

Construction Content Generator

Course Name:Gen AI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	SNEHA GUPTA	EN22CS306051
2	NAMAN SUNDRIYAL	EN22CS304042
3	AKASH SINGH	EN22CS303009
4	YASH DHIMOLE	EN22CS304066
5	HARSH NANDWAL	EN22CS304027
6	SHUBH RATHOD	EN22CS304060

Group Name: Group 12D11

Project Number: GAI-38

Industry Mentor Name: Mr. Rishabh Tripathi

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the document

This document describes the **High Level Design (HLD)** of the **Construction Content Generator** system. The purpose of this document is to provide a broad architectural view of the solution, including its main modules, data handling approach, interfaces, and major design considerations.

The system is designed to generate professional construction documents such as **Daily Site Reports, Weekly Progress Reports, Safety Reports, and Material Summaries** using **Python, Vector DB, and an LLM API**.

This document focuses on:

- overall system structure
- major components and their interaction
- high-level data handling
- interfaces between modules
- non-functional expectations of the system

1.2 Intended Audience

This document is intended for:

- project evaluators and faculty members
- developers and team members
- system designers and reviewers
- students preparing project presentations and viva
- future contributors extending the project

It provides a high-level understanding of how the system is organized and how its main components work together.

1.3 System overview

The **Construction Content Generator** is a domain-specific AI solution that helps construction professionals automate repetitive documentation tasks. Instead of manually drafting reports, users provide structured construction-related inputs such as project details, work progress, safety notes, and issues.

The system then:

1. processes the input using Python
2. retrieves relevant construction knowledge from a Vector Database
3. combines this context with user input
4. generates a structured construction document using an LLM API

The solution follows a **Retrieval-Augmented Generation (RAG)** approach to improve output relevance, structure, and terminology consistency.

Supported outputs include:

- Daily Site Report
- Weekly Progress Report
- Safety Report
- Material Summary

2. System Design

2.1 Application Design

The application is designed as a **modular AI-powered backend system**. The overall design separates the solution into distinct functional layers so that each major responsibility is isolated and manageable.

High-level application layers

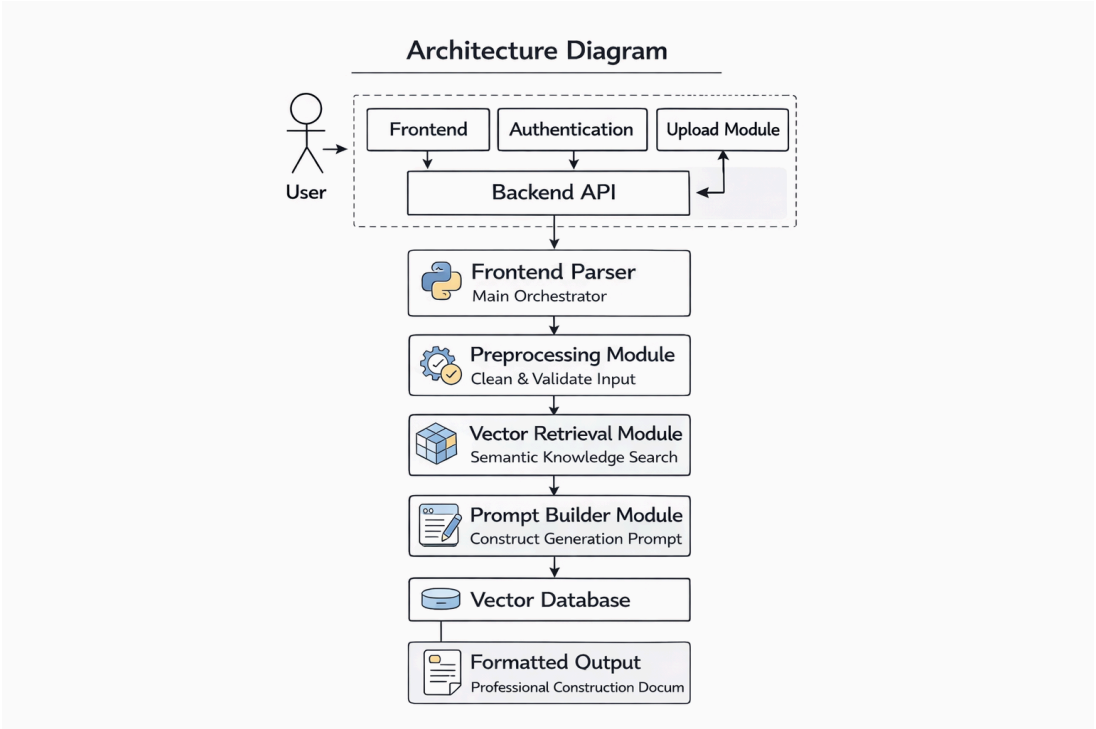
- **Input Layer**
  Accepts structured construction details from the user.
- **Processing Layer**
  Cleans, validates, and organizes user input.

- **Retrieval Layer**
  Searches the Vector Database for relevant templates, terminology, and construction knowledge.
- **Prompt Construction Layer**
  Builds the final prompt using user input and retrieved context.
- **Generation Layer**
  Uses the LLM API to generate the final construction document.
- **Output Layer**
  Returns the professional formatted report to the user.

Design approach

The system is designed to be:

- modular
- maintainable
- scalable
- domain-specific
- easily extensible for future document types



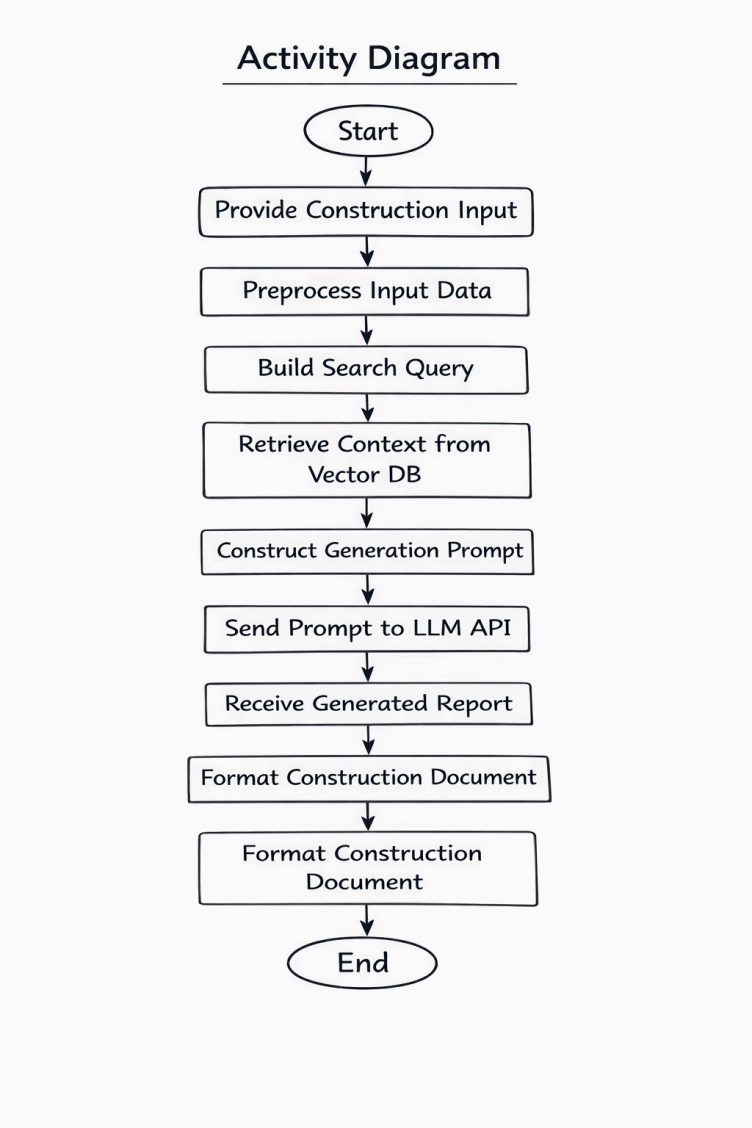
2.2 Process Flow

The overall process flow is as follows:

1. User provides construction-related details.
2. The system receives the input in structured format.
3. Python preprocesses and validates the input.
4. A search query is generated from important user fields.
5. The query is sent to the Vector DB.
6. The Vector DB returns relevant construction knowledge.
7. Python combines user input and retrieved context into a generation prompt.
8. The prompt is sent to the LLM API.
9. The LLM API generates the final document.
10. The system formats and returns the output.

High-level workflow

User Input → Processing → Retrieval → Prompt Construction → LLM Generation → Output



2.3 Information Flow

The information flow within the system is divided into three major phases:

1. Input Information Flow

The user provides structured construction data such as:

- project name
- location
- date
- document type

- work completed
- safety observations
- issues and delays
- next plan

This data is captured as structured input and passed to the backend processing layer.

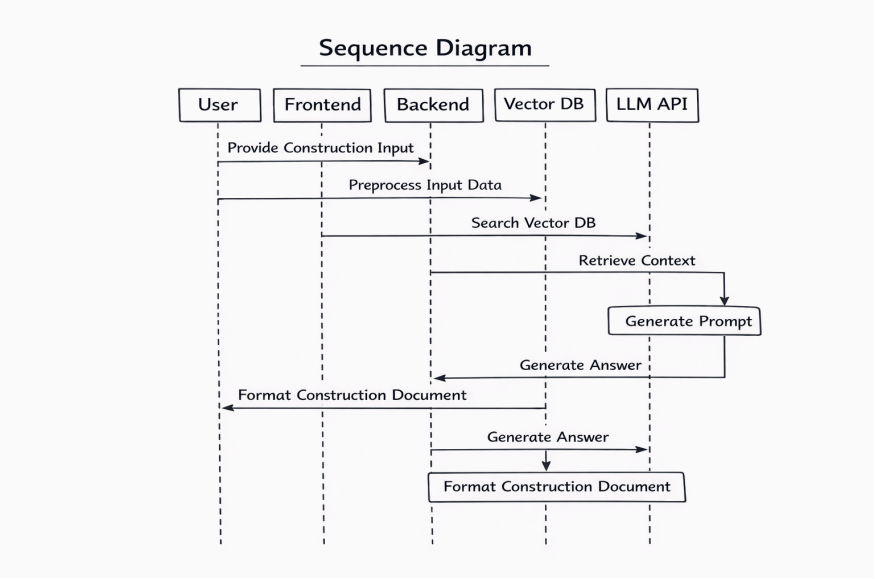
2. Internal Information Flow

- Python receives the structured data.
- Relevant fields are selected for query construction.
- The query is passed to the Vector DB.
- Retrieved knowledge is sent back to Python.
- Python builds the prompt for the LLM API.
- LLM API generates report content.

3. Output Information Flow

The generated text is returned as:

- structured construction document
- professional report format
- domain-specific content output



2.4 Components Design

1. User Input Component

This component is responsible for capturing structured construction-related information from the user. It acts as the entry point of the system.

2. Python Processing Component

This is the core orchestration component of the application. It handles:

- input validation
- preprocessing
- query generation
- prompt generation
- response handling

3. Vector Database Component

This component stores construction knowledge such as:

- report templates
- construction terminology
- safety guidelines
- documentation standards

It provides semantic retrieval of relevant information based on the query.

4. Prompt Builder Component

This component combines:

- cleaned user input
- retrieved Vector DB context
- formatting rules
- generation instructions

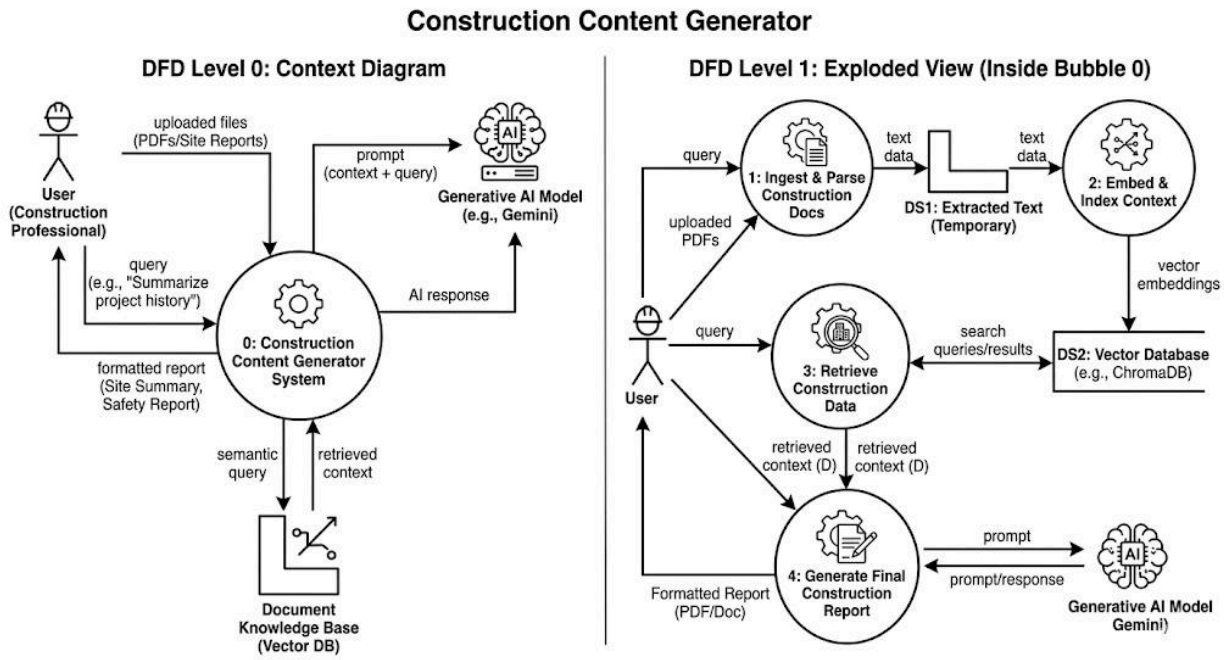
It prepares the final prompt that will be sent to the LLM.

5. LLM API Component

This component is responsible for generating the final construction document. It receives the constructed prompt and returns structured professional content.

6. Output Component

This component formats and presents the generated report in readable form.



2.5 Key Design Considerations

1. Domain-Specific Accuracy

The system should generate construction-related content using proper terminology and report structure.

2. Retrieval-Augmented Generation

Using Vector DB before content generation improves relevance and reduces generic responses.

3. Modularity

The system is divided into separate logical components so future changes can be made easily.

4. Extensibility

New report types can be supported by adding:

- new prompt templates
- new knowledge documents
- new generation rules

5. Reduced Hallucination

The design ensures missing values are treated carefully and domain knowledge is retrieved before generation.

6. Maintainability

Each component has a clear responsibility, making testing and debugging easier.

7. Usability

The system should support straightforward input and produce clearly formatted output.

3. Data Design

3.1 Data Model

The system works with two main categories of data:

1. User Input Data

This includes structured report fields such as:

- document type
- project details
- work details

- manpower
- materials
- equipment
- safety notes
- issues
- next plan

2. Knowledge Base Data

This includes construction-related reference content such as:

- report templates
- safety guidelines
- terminology lists
- standard reporting phrases

High-level data entities

- User Input Entity
- Search Query Entity
- Retrieved Context Entity
- Prompt Entity
- Generated Report Entity

3.2 Data Access Mechanism

The system accesses data in the following way:

User Data Access

User-provided structured fields are received and handled by Python.

Knowledge Access

The Vector DB stores embedded construction-related documents. Python queries this database using a search query and retrieves relevant content.

Generated Output Access

The generated report is returned directly from the LLM API response and passed to the output component.

3.3 Data Retention Policies

For the current version of the project:

- user input is processed only for report generation
- knowledge base documents remain stored in the Vector DB
- generated reports may be displayed or optionally stored if future persistence is added

Retention approach

- static knowledge is retained in Vector DB
- dynamic user input is temporary unless future storage is introduced
- generated reports are session-level outputs in the present design

3.4 Data Migration

At the current project scope, data migration is minimal because:

- the knowledge base is ingested into the Vector DB from prepared text files
- updates to templates or terminology can be handled by re-ingesting or updating the stored documents

Possible migration scenarios

- adding new template files
- updating safety guidelines
- extending terminology datasets
- migrating knowledge base to a larger or cloud-based Vector DB in the future

4. Interfaces

The system has the following major interfaces:

1. User-to-System Interface

The user provides construction input in structured format.

2. Python-to-Vector DB Interface

Python sends the search query to the Vector DB and receives retrieved context.

3. Python-to-LLM API Interface

Python sends the final prompt to the LLM API and receives the generated report.

4. Internal Module Interfaces

Internal components communicate through Python functions and structured data objects.

Interface summary

- input interface
- retrieval interface
- generation interface
- output interface

5. State and Session Management

The current design is mostly **stateless** in its generation workflow.

State handling

- each report generation request is processed independently
- user input is handled per request
- generated output is produced for the current session/request

Session handling

At the current scope:

- long-term session persistence is not required
- no login-based user session is mandatory
- future enhancements can add report history and saved sessions

This makes the current design simpler and easier to implement.

6. Caching

The present version of the system does not require advanced caching, but caching can improve performance in the future.

Potential caching opportunities

- repeated retrieval results for common document types
- reused prompt templates
- frequently accessed knowledge chunks
- repeated LLM responses for identical queries in demo environments

Current approach

The project can function without explicit caching in its initial implementation.

7. Non-Functional Requirements

7.1 Security Aspects

The system should follow basic security practices such as:

- storing API keys securely
- avoiding hardcoded credentials
- validating user input before processing
- controlling access to knowledge base files
- preventing malformed or empty prompt submissions

For the current project scope, basic secure configuration is sufficient.

7.2 Performance Aspects

The system should provide acceptable performance for report generation.

Performance expectations

- input handling should be fast
- Vector DB retrieval should return top relevant chunks quickly
- LLM response time should be reasonable for report generation
- overall workflow should remain responsive for small to medium-sized knowledge bases

Performance design choices

- use limited top results from Vector DB
- keep prompts concise
- store domain knowledge in organized form
- separate retrieval and generation for clarity and control

8. References

1. Python documentation
2. Vector Database documentation
3. LLM API documentation
4. Construction reporting standards and sample templates
5. Prompt engineering concepts for structured AI output